

PersonalAI — 30 Fresher GenAI Interview Questions & Answers

A comprehensive technical preparation guide covering RAG, FAISS, Embeddings, FastAPI, SQLite, and Transformer concepts.

Q1: What is Retrieval-Augmented Generation (RAG)?

Simple Answer: RAG is a technique where an AI model searches external private documents for relevant information before generating an answer, ensuring accurate and up-to-date responses.

Technical Explanation: RAG decouples information storage from parametric model weights. User queries are embedded into vector space, searched against a vector database (FAISS) via cosine similarity, and top- K chunks are injected into the LLM system prompt as explicit context.

Small Example:

Q2: Why did you use RAG instead of Fine-Tuning an LLM?

Simple Answer: RAG allows adding or updating documents instantly without spending money on retraining, and it prevents hallucinations by providing source citations.

Technical Explanation: Fine-tuning modifies internal weights to adapt tone or domain syntax, but is inefficient for factual recall on dynamic data. Fine-tuning suffers from knowledge decay, high compute costs, and catastrophic forgetting. RAG offers $O(1)$ dynamic knowledge updates.

Small Example:

Q3: What are Embeddings and how are they generated?

Simple Answer: Embeddings are arrays of numbers that represent the meaning of words or sentences so computers can compare their similarity.

Technical Explanation: An embedding function maps discrete tokens into a continuous vector space $\mathbb{R}^d$ (e.g., $d=1536$ for OpenAI `text-embedding-3-small`). Deep neural networks trained on metric learning loss maximize dot product for semantically similar pairs.

Small Example:

Q4: How does Cosine Similarity work in Vector Search?

Simple Answer: Cosine similarity measures the angle between two vector arrows in space. A smaller angle means closer mathematical alignment.

Technical Explanation:

Small Example:

Q5: What is FAISS and why did you select it?

Simple Answer: FAISS is a high-speed library developed by Meta for searching vector similarity locally without needing cloud vector databases.

Technical Explanation: FAISS (Facebook AI Similarity Search) builds fast vector indexes (e.g., `IndexFlatIP`, `IndexIVFFlat`, `HNSW`). For a fresher portfolio project, `IndexFlatIP` on L_2 -normalized vectors offers exact,

zero-latency cosine similarity search without SaaS cost or network bottlenecks.

Small Example:

Q6: What is Chunking and why is it necessary?

Simple Answer: Chunking splits long documents into smaller readable sections so they fit inside the LLM's context window and improve search accuracy.

Technical Explanation: Whole documents dilute vector embedding density and exceed token limits. Splitting text into fixed size chunks (e.g., 500 characters) preserves focused semantic concepts per chunk, maximizing retrieval precision.

Small Example:

Q7: Why do we use Chunk Overlap?

Simple Answer: Chunk overlap ensures key sentences split at boundary edges do not lose context between adjacent chunks.

Technical Explanation: Without overlap, a critical phrase spanning character boundary `$N$` (e.g., "The candidate graduated from... | ...Stanford University in 2024") gets sliced, degrading embedding quality for both chunks.

Small Example:

Q8: What is Tokenization and how does tiktoken work?

Simple Answer: Tokenization converts raw text strings into numerical token IDs that the LLM understands.

Technical Explanation: OpenAI uses Byte-Pair Encoding (BPE) via the ``tiktoken`` library. Common words are single tokens, while rare words split into sub-word tokens.

Small Example:

Q9: How does RAG control Hallucination?

Simple Answer: We instruct the prompt to answer only using the provided retrieved context, and explicitly say "I don't know" if facts are absent.

Technical Explanation: By placing system guardrails and strict negative constraints in the system prompt ("If context does not contain the answer, reply 'Information not found'"), we constrain the autoregressive generation decoding space.

Small Example:

Q10: What is the Transformer Architecture and Self-Attention?

Simple Answer: A Transformer is a neural network architecture that processes all words simultaneously, using attention to determine how words relate to each other.

Technical Explanation: Introduced in 2017, Transformers compute scaled dot-product attention:

Small Example:

Q11: What is FastAPI and why use it over Flask or Django?

Simple Answer: FastAPI is a modern, ultra-fast Python web framework built on ASGI with automatic type verification and interactive Swagger API documentation.

Technical Explanation: Built on Starlette and Pydantic, FastAPI leverages Python ``async/await`` for high-concurrency non-blocking I/O. It auto-generates OpenAPI JSON schemas at ``/docs``.

Small Example:

Q12: Why use SQLite + SQLAlchemy for this project?

Simple Answer: SQLite provides lightweight, zero-configuration local database storage, while SQLAlchemy provides a clean Python object interface.

Technical Explanation: SQLAlchemy ORM provides ACID transactions, relational schema mappings (`User`, `Document`, `Conversation`, `Message`), and foreign key cascade deletions.

Small Example:

Q13: How are NumPy and Pandas used in PersonalAI?

Simple Answer: Pandas calculates document tabular statistics and summary reports; NumPy handles vector math like cosine similarity.

Technical Explanation:

Small Example:

Q14: What happens end-to-end when a user uploads a document?

Simple Answer: Text is extracted, cleaned, chunked into pieces, converted to vector embeddings, stored in FAISS, and file stats are saved in SQLite.

Technical Explanation:

Q15: What happens end-to-end when a user asks a question?

Simple Answer: Question $\rightarrow$ Embedding $\rightarrow$ FAISS Search $\rightarrow$ Context Prompt $\rightarrow$ OpenAI LLM $\rightarrow$ Answer + Sources.

Technical Explanation:

Q16: How does PersonalAI handle User Personalization?

Simple Answer: The user's name, skills, education, and preferred response style are saved in SQLite and injected into every RAG prompt.

Technical Explanation: The system prompt contains dynamic placeholders `{user\_name}`, `{user\_skills}`, `{user\_response\_style}` which condition the LLM's system persona.

Q17: What is the purpose of memory management in Chatbot sessions?

Simple Answer: We store chat history in SQLite but only send the last few messages to the LLM to avoid exceeding token limits or incurring high costs.

Technical Explanation: `MemoryService` implements a sliding window context strategy (`limit=6`), trimming older messages while retaining session continuity.

Q18: How do you prevent exposing API keys in a public GitHub repository?

Simple Answer: API keys are stored in a `.env` file that is ignored by Git using `.gitignore`.

Technical Explanation: Secrets are loaded runtime via `pydantic-settings` or `python-dotenv`. `.env.example` provides template keys without real credentials.

Q19: How does Document Summarization work in PersonalAI?

Simple Answer: The user selects a document, and the LLM processes its text with prompts for a short summary, detailed summary, or bulleted key points.

Technical Explanation: ``DocumentService`` reads document text, truncates to token safety limits, and feeds it into ``SUMMARIZATION_PROMPT``.

Q20: How does the Resume Analyzer extract structured insights?

Simple Answer: It feeds the resume text into an LLM prompt that formats outputs into education, skills, technologies, and project sections.

Technical Explanation: ``RESUME_ANALYSIS_PROMPT`` enforces JSON-like key extraction for targeted resume query resolution.

Q21: How does the Job Description Matcher perform gap analysis?

Simple Answer: It compares the candidate's skills and uploaded resume with a target job description and highlights matching vs missing skills.

Technical Explanation: ``JOB_MATCH_PROMPT`` receives both Candidate Profile context and Job Description text, prompting the LLM to output structured skill matrices and interview topics.

Q22: What are the limitations of FAISS in production?

Simple Answer: FAISS runs in local server memory; if the server restarts without persistence, vectors must be re-loaded, and it lacks multi-node scaling out-of-the-box.

Technical Explanation: FAISS is an in-process C++ library without built-in RPC APIs, user access control, or live horizontal sharding.

Q23: How would you replace FAISS with a cloud Vector DB like Pinecone or Qdrant?

Simple Answer: Replace ``vector_store.py`` with the Pinecone SDK client calls (``index.upsert()`` and ``index.query()``) while keeping the rest of the application unchanged.

Technical Explanation: Because of modular service separation (``FAISSVectorStore`` encapsulation), swapping to Pinecone only requires re-implementing ``add_chunks()``, ``search()``, and ``delete_document_chunks()``.

Q24: What is System Prompt vs User Prompt in OpenAI API?

Simple Answer: System prompt sets the rules, background persona, and constraints; User prompt contains the actual question or input.

Technical Explanation: In ChatML format, ``{"role": "system"}`` establishes high-priority context and guardrails, whereas ``{"role": "user"}`` supplies conversational inputs.

Q25: Why is Streamlit ideal for Generative AI prototyping?

Simple Answer: Streamlit turns Python scripts into interactive web dashboards instantly without writing HTML, CSS, or JavaScript.

Technical Explanation: Streamlit re-executes Python scripts on state changes, binding reactive widgets (``st.chat_input``, ``st.dataframe``) directly to backend API calls.

Q26: What is Temperature in OpenAI LLM configuration?

Simple Answer: Temperature controls randomness in output. Lower temperature means deterministic, factual answers; higher temperature means creative answers.

Technical Explanation: Temperature scales the logits in the softmax distribution:

Q27: How do you handle file size limits and validation security?

Simple Answer: We validate file extensions (`pdf`, `docx`, `txt`, `md`) and enforce a 15MB file size check before processing.

Technical Explanation: Checked in FastAPI endpoint using `UploadFile.filename` extension whitelist and file size verification on disk.

Q28: How would you scale PersonalAI for thousands of concurrent users?

Simple Answer: Move database to PostgreSQL, vector store to Pinecone/Qdrant, run FastAPI workers with Uvicorn behind Nginx, and host Streamlit on cloud servers.

Technical Explanation:

Q29: What is the role of Git and GitHub in this project?

Simple Answer: Git tracks code changes locally; GitHub hosts the code publicly for portfolio presentation and code collaboration.

Technical Explanation: Using feature branches, commit messages, `.gitignore` rules for secrets, and `README.md` documentation.

Q30: What future improvements would you add to PersonalAI?

Simple Answer: Add multi-modal OCR for scanned PDFs, hybrid search (keyword + vector), re-ranking (Cohere Rerank), and voice Q&A; capabilities.

Technical Explanation: